Single Restaurant Food Delivery App - Complete Plan

1. System Overview

User Roles (Simplified)

- **Customers**: Browse menu, place orders, track delivery
- **Restaurant Staff**: Manage orders, update menu, view analytics
- **Delivery Drivers**: Accept deliveries, update location, mark completed
- **Admin/Owner**: Manage everything, view reports

Core Components

- Customer Mobile/Web App
- Restaurant Management Dashboard (Web)
- Delivery Driver Mobile App
- Backend API Server
- Real-time tracking service
- Payment processing
- Notification system

2. Technology Stack

Frontend

- **Customer App**: React.js (Web) + Progressive Web App (PWA) for mobile
  - Or React Native if you want native mobile apps
- **Restaurant Dashboard**: React.js with admin UI library (Material-UI or Ant Design)
- **Driver App**: React Native or Flutter for iOS/Android

Backend

- **API Server**: Node.js with Express.js (easier for real-time features)
- **Database**: PostgreSQL for all data
- **Cache/Session**: Redis for real-time tracking and sessions
- **File Storage**: AWS S3 or Cloudinary for food images

Real-time Features

- **WebSockets**: Socket.io for live order updates and tracking
- **Maps**: Google Maps API or Mapbox for tracking and navigation
- **Geolocation**: HTML5 Geolocation API

Infrastructure

- **Hosting**: AWS, DigitalOcean, or Heroku (start simple)
- **CDN**: Cloudflare for images and static files
- **Domain & SSL**: Your restaurant domain with HTTPS

Payment

- **Payment Gateway**: Stripe or Razorpay (India)
- Support: Cards, UPI, Wallets, Cash on Delivery

Notifications

- **Push Notifications**: Firebase Cloud Messaging (FCM)
- **SMS**: Twilio or MSG91 (for India)
- **Email**: SendGrid or AWS SES

3. Database Schema

Tables You Need:

customers

```
- id (primary key)
- name
- email
- phone
- password_hash
- created_at
```

customer_addresses

- id
- customer_id (foreign key)
- address_line
- landmark
- city
- pincode
- latitude
- longitude
- is_default

menu_categories

- id
- name (e.g., "Starters", "Main Course", "Desserts")
- display_order
- is_active

menu_items

- id
- category_id
- name
- description
- price
- image_url
- is_vegetarian
- is_available
- prep_time_minutes
- created_at

orders

- id
- customer_id
- delivery_address_id
- items (JSON array with item_id, quantity, price)
- subtotal
- delivery_fee
- taxes
- discount_amount
- total_amount
- payment_method
- payment_status (pending/completed/failed)
- order_status (placed/confirmed/preparing/ready/out_for_delivery/delivered/cancelled)
- special_instructions
- driver_id
- placed_at
- estimated_delivery_time
- delivered_at

delivery_drivers

- id
- name
- phone
- vehicle_type
- vehicle_number
- is_available
- current_latitude
- current_longitude
- last_location_update
- created_at

order_tracking

- id
- order_id
- driver_id
- location_updates (JSON array of {lat, lng, timestamp})
- status_history (JSON array of {status, timestamp})

restaurant_settings

- id
- restaurant_name
- address
- phone
- email
- opening_time
- closing_time
- is_accepting_orders
- delivery_radius_km
- minimum_order_amount
- delivery_fee
- estimated_prep_time_minutes

4. Feature Breakdown

PHASE 1: MVP (8-10 weeks)

Customer App Features:

- [x] User registration/login (phone or email)
- [x] Browse menu by categories
- [x] View item details with images
- [x] Add to cart with quantity selection
- [x] Manage delivery addresses
- [x] Apply promo codes/discounts
- [x] Choose payment method
- [x] Place order
- [x] View order status (text-based updates)
- [x] Order history
- [x] Basic profile management

Restaurant Dashboard:

- [x] Login for staff/admin
- [x] Real-time incoming order notifications
- [x] Accept/reject orders
- [x] Update order status (preparing → ready → out for delivery)
- [x] Menu management (add/edit/delete items)
- [x] Toggle item availability
- [x] Update restaurant settings (open/closed, delivery fee)
- [x] View today's orders
- [x] Basic sales report

Driver App:

- [x] Driver login
- [x] View assigned deliveries
- [x] Accept delivery assignment
- [x] View delivery address and customer phone
- [x] Navigation to customer location
- [x] Update delivery status
- [x] Mark order as delivered
- [x] View delivery history

Backend:

- [x] User authentication (JWT tokens)
- [x] CRUD APIs for menu, orders, users
- [x] Order placement logic
- [x] Payment integration (basic)
- [x] Order status management
- [x] Driver assignment (manual or automatic)

PHASE 2: Live Tracking (3-4 weeks)

Customer App:

- [x] Live map showing driver's location
- [x] Real-time ETA updates
- [x] Driver details (name, phone, vehicle)
- [x] Call driver button

Driver App:

- [x] Automatic location sharing (every 10-15 seconds)
- [x] Route optimization
- [x] Distance to customer

Backend:

- [x] WebSocket server for real-time updates
- [x] Location tracking system
- [x] ETA calculation using Google Maps Distance Matrix
- [x] Push location updates to customers

PHASE 3: Enhanced Features (4-6 weeks)

- Push notifications for order updates
- SMS notifications

- Rating and review system
- Favorite items
- Reorder from history
- Scheduled orders (order for later)
- Multiple payment methods
- Digital wallet/credits
- Promo code management system
- Customer support chat
- Driver performance analytics
- Sales analytics dashboard

5. Real-Time Tracking - Technical Details

How It Works:

Step 1: Order Assignment

1. Restaurant accepts order and assigns to driver
2. Driver accepts the delivery
3. System creates tracking session

Step 2: Location Updates

1. Driver app requests location permission
2. App sends GPS coordinates every 10-15 seconds to backend
3. Endpoint: `POST /api/tracking/update-location`

```json
{
  "driver_id": "123",
  "order_id": "456",
  "latitude": 28.5355,
  "longitude": 77.3910,
  "timestamp": "2025-10-18T10:30:45Z"
}
```

Step 3: Broadcasting to Customer

1. Customer app connects to WebSocket when order is "out_for_delivery"

2. Backend broadcasts location updates via Socket.io
3. Customer sees driver moving on map in real-time

Step 4: ETA Calculation

1. Every 30 seconds, backend calculates distance using Google Maps API
2. Gets current traffic conditions
3. Updates estimated arrival time
4. Pushes to customer app

Technical Architecture:

```
Driver App (GPS)
    ↓
REST API (/api/tracking/update-location)
    ↓
Redis Cache (fast storage)
    ↓
Socket.io Server (WebSocket)
    ↓
Customer App (Map Display)
```

Code Flow Example:

Driver sends location:

```javascript
// Driver app (every 15 seconds)
const location = await getCurrentPosition();
await fetch('/api/tracking/update-location', {
  method: 'POST',
  body: JSON.stringify({
    order_id: currentOrderId,
    latitude: location.coords.latitude,
    longitude: location.coords.longitude
  })
});
```

Backend stores and broadcasts:

```javascript
```

```
// Backend
app.post('/api/tracking/update-location', async (req, res) => {
  const { order_id, latitude, longitude } = req.body;

  // Store in Redis for fast access
  await redis.set(`order:${order_id}:location`,
    JSON.stringify({ latitude, longitude, timestamp: Date.now() })
  );

  // Calculate ETA
  const eta = await calculateETA(latitude, longitude, customerAddress);

  // Broadcast to customer via WebSocket
  io.to(`order:${order_id}`).emit('location_update', {
    latitude,
    longitude,
    eta
  });

  res.json({ success: true });
});
```

Customer receives updates:

```
javascript

// Customer app
socket.on('location_update', (data) => {
  updateMapMarker(data.latitude, data.longitude);
  updateETA(data.eta);
});
```

6. Development Timeline

Week 1-2: Setup & Infrastructure

- Set up GitHub repositories
- Configure development environment
- Set up database (PostgreSQL)
- Set up Redis
- Create basic project structure for all apps
- Design UI/UX mockups

Week 3-4: Backend Core

- User authentication system
- Menu CRUD APIs
- Order placement API
- Payment gateway integration
- Database migrations

Week 5-6: Customer App (Basic)

- Menu browsing UI
- Cart functionality
- Checkout flow
- Payment integration
- Order placement

Week 7-8: Restaurant Dashboard

- Login system
- Order management UI
- Menu management
- Real-time order notifications
- Basic analytics

Week 9-10: Driver App (Basic)

- Driver authentication
- View assigned orders
- Basic navigation
- Update order status
- Delivery completion

Week 11-12: Live Tracking

- Location tracking in driver app
- WebSocket server setup
- Real-time map in customer app

- ETA calculation
- Testing and optimization

Week 13-14: Testing & Polish

- End-to-end testing
- Bug fixes
- Performance optimization
- UI/UX improvements
- Security testing

Week 15-16: Launch Preparation

- Deploy to production servers
- Set up monitoring and logging
- Train restaurant staff
- Onboard delivery drivers
- Soft launch with test customers

7. Cost Estimation

Development (DIY vs Hiring):

Option A: Hire Development Team

- 2 Full-stack developers: $15k-25k (4 months)
- 1 UI/UX designer: $5k-8k
- 1 Mobile developer: $10k-15k **Total: $30k-48k**

Option B: Freelancers

- Full project: $10k-20k (varies by location/quality)

Option C: Development Agency

- Complete solution: $25k-50k

Monthly Operational Costs:

Infrastructure:

- Server hosting (DigitalOcean/AWS): $50-200/month
- Database: Included or $20-50/month

- Redis: $10-30/month
- CDN/Storage: $10-30/month

APIs & Services:

- Google Maps API: $50-300/month (based on usage)
- Payment gateway: 2-3% per transaction
- SMS service: $20-100/month
- Push notifications: Free (Firebase) or $10-50/month

Domain & SSL:

- Domain: $10-20/year
- SSL: Free (Let's Encrypt)

Total Monthly: $150-500/month (can scale up as you grow)

8. MVP Launch Checklist

Before Launch:

- [ ] Customer app tested on iOS and Android
- [ ] Menu uploaded with quality images
- [ ] Payment gateway in production mode
- [ ] At least 3 delivery drivers onboarded and trained
- [ ] Restaurant staff trained on dashboard
- [ ] Test orders completed successfully
- [ ] Live tracking tested in real conditions
- [ ] Notification system working (push + SMS)
- [ ] Terms of service and privacy policy pages
- [ ] Customer support phone/email set up
- [ ] Backup system for orders (manual fallback)
- [ ] Monitoring and error logging configured

Marketing Material:

- [ ] App store listing (iOS)
- [ ] Play store listing (Android)
- [ ] Social media presence
- [ ] Promotional materials (flyers, posters)
- [ ] Launch discount/offer ready

9. Simplified Starter Version

If you want to start even simpler, here's a 4-week MVP:

Week 1-2: Web-Only Version

- Simple website with menu
- WhatsApp integration for orders
- Manual tracking (SMS updates)
- Cash on delivery only

Week 3-4: Basic Customer App

- View menu and place orders
- Online payment
- Order status page (no map)
- Staff dashboard for order management

Then add tracking later as you validate the concept and get customers.

10. Key Success Factors

Technical:

- Keep initial version simple and stable
- Focus on fast order processing (under 5 seconds)
- Ensure app works on low-end phones
- Offline support for driver app
- Reliable notifications

Business:

- Fast delivery times (under 30 minutes in your area)
- Quality food packaging
- Professional drivers
- Responsive customer support
- Regular menu updates and offers

Marketing:

- Launch in small area first
- Offer attractive launch discounts

- Collect customer feedback actively
- Build loyalty program early
- Leverage social media

11. Alternatives to Custom Development

If budget is tight, consider:

Option 1: White-label Solutions

- Companies like UpMenu, GloriaFood, or ChowNow
- Cost: $50-300/month
- Pros: Quick setup, maintained by provider
- Cons: Less customization, monthly fees

Option 2: WordPress/WooCommerce Plugin

- Food ordering plugins available
- Cost: $50-200 one-time + hosting
- Pros: Cheaper, customizable
- Cons: Less sophisticated, may need developer

Option 3: No-Code Platforms

- Bubble.io, Adalo, Glide
- Cost: $0-100/month
- Pros: Build yourself, no coding
- Cons: Limited features, scaling issues

Custom development gives you:

- Full control and ownership
- Unique features and branding
- Better integration with your operations
- No recurring license fees
- Ability to scale and add features

12. Post-Launch Roadmap

Month 1-3:

- Gather customer feedback
- Fix bugs and improve UX
- Optimize delivery routes
- Add payment methods
- Introduce loyalty program

Month 4-6:

- Add scheduling feature
- Build customer referral program
- Create promotional campaigns tool
- Add ratings and reviews
- Implement analytics dashboard

Month 7-12:

- AI-based recommendations
- Group ordering feature
- Subscription plans
- Expand delivery area
- Open second location integration

Next Steps: Would you like me to help you with:

1. Detailed API documentation?
2. Database schema SQL scripts?
3. A prototype of the customer app?
4. The real-time tracking implementation code?